

PART 1: MINIMAX ALGORITHM

1. Introduction

Minimax is a **search algorithm** used in **Artificial Intelligence** for **decision making in adversarial environments**.

It is mainly used in:

- Two-player games
- Turn-based games
- Zero-sum games (one player's gain is the other player's loss)

Examples include:

- Chess
- Checkers
- Connect Four
- Tic-Tac-Toe

Minimax assumes that:

- The opponent is intelligent.
- The opponent will always make the best possible move

2. Basic Idea of Minimax

The name comes from:

- MIN → minimizing player
- MAX → maximizing player

There are two players:

- MAX tries to maximize the score.
- MIN tries to minimize the score.

The algorithm explores all possible future game states and chooses the move that gives the best guaranteed outcome

3. Game Tree Representation

Minimax works using a **game tree**.

- Each node represents a board state.
- Edges represent possible moves.

- Leaf nodes represent terminal states (win, loss, draw).

Levels alternate between:

- MAX nodes (AI turn)
- MIN nodes (Opponent turn)

Example scoring in Tic-Tac-Toe:

- AI win $\rightarrow +1$
- Human win $\rightarrow -1$
- Draw $\rightarrow 0$

4. Working of Minimax Step-by-Step

1. Generate all possible moves.
2. Recursively explore each move.
3. Evaluate terminal states.
4. Backtrack values upward.
5. MAX chooses highest value.
6. MIN chooses lowest value.

The final selected move is the one that maximizes the minimum possible loss.

5. Mathematical Expression

Minimax can be written as:

MAX over all moves of (MIN over opponent moves of Value)

This means:

The algorithm chooses the move that gives the best worst-case result.

7. Algorithm

FUNCTION MINIMAX-DECISION(STATE) RETURNS AN ACTION

 RETURN $\operatorname{argmax}_{a \in \text{ACTIONS}(\text{STATE})} \text{MIN-VALUE}(\text{RESULT}(\text{STATE}, a))$

FUNCTION MAX-VALUE(STATE) RETURNS A UTILITY VALUE

 IF $\text{TERMINAL-TEST}(\text{STATE})$ THEN

 RETURN $\text{UTILITY}(\text{STATE})$

$v \leftarrow -\infty$

 FOR EACH ACTION a IN $\text{ACTIONS}(\text{STATE})$ DO

$v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(\text{STATE}, a)))$

RETURN v

FUNCTION MIN-VALUE(STATE) RETURNS A UTILITY VALUE

IF TERMINAL-TEST(STATE) THEN

RETURN UTILITY(STATE)

$v \leftarrow +\infty$

FOR EACH ACTION a IN ACTIONS(STATE) DO

$v \leftarrow \min(v, \text{MAX-VALUE}(\text{RESULT}(\text{STATE}, a)))$

RETURN v

7. Time and Space Complexity

Time Complexity:

$O(b^d)$

Where:

- b = branching factor
- d = depth of tree

Space Complexity:

$O(d)$ (due to recursion stack)

For large games like chess, this becomes computationally expensive.

8. Limitations of Minimax

- Explores entire game tree.
- Exponential time growth.
- Not practical for large state spaces without optimization.
- Does not handle randomness.
- Requires evaluation function for deep games.

9. Example Code – Minimax

Python code:

```
import tkinter as tk
from tkinter import messagebox
import math

board = [" " for _ in range(9)]
human = "X"
```

```

ai = "0"

def check_winner(player):
    wins = [
        [0,1,2],[3,4,5],[6,7,8],
        [0,3,6],[1,4,7],[2,5,8],
        [0,4,8],[2,4,6]
    ]
    for w in wins:
        if board[w[0]] == board[w[1]] == board[w[2]] == player:
            return True
    return False

def is_full():
    return " " not in board

def minimax(is_maximizing):
    if check_winner(ai):
        return 1
    if check_winner(human):
        return -1
    if is_full():
        return 0

    if is_maximizing:
        best = -math.inf
        for i in range(9):
            if board[i] == " ":
                board[i] = ai
                score = minimax(False)
                board[i] = " "
                best = max(best, score)
        return best
    else:
        best = math.inf
        for i in range(9):
            if board[i] == " ":
                board[i] = human
                score = minimax(True)
                board[i] = " "
                best = min(best, score)
        return best

def best_move():
    best_score = -math.inf
    move = None
    for i in range(9):
        if board[i] == " ":

```

```

        board[i] = ai
        score = minimax(False)
        board[i] = " "
        if score > best_score:
            best_score = score
            move = i
    return move

def click(i):
    if board[i] == " ":
        board[i] = human
        buttons[i]["text"] = human

        if check_winner(human):
            messagebox.showinfo("Game Over", "You Win!")
            root.destroy()
            return

        if is_full():
            messagebox.showinfo("Game Over", "Draw!")
            root.destroy()
            return

        ai_move = best_move()
        if ai_move is not None:
            board[ai_move] = ai
            buttons[ai_move]["text"] = ai

            if check_winner(ai):
                messagebox.showinfo("Game Over", "AI Wins!")
                root.destroy()
                return

            if is_full():
                messagebox.showinfo("Game Over", "Draw!")
                root.destroy()

root = tk.Tk()
root.title("Tic Tac Toe - Minimax")

buttons = []
for i in range(9):
    btn = tk.Button(root, text=" ", font=("Arial", 30),
                    width=5, height=2,
                    command=lambda i=i: click(i))
    btn.grid(row=i//3, column=i%3)
    buttons.append(btn)

```

```
root.mainloop()
```

PART 2: ALPHA-BETA PRUNING

1. Introduction

Alpha-Beta Pruning is an optimization technique applied to Minimax.

It reduces the number of nodes evaluated in the game tree without affecting the final decision.

Important point:

Alpha-Beta produces the same result as Minimax but faster.

2. Core Idea

During tree traversal, some branches are guaranteed not to influence the final decision.

Such branches are pruned (ignored).

Two values are maintained:

Alpha (α):

- Best value that MAX can guarantee so far.

Beta (β):

- Best value that MIN can guarantee so far.

If at any time:

$$\alpha \geq \beta$$

Further exploration of that branch stops.

3. Why Pruning Works

Suppose:

- MAX already found a move with value 5.
- Now evaluating another branch.
- MIN finds a move with value 3.

Since MIN will choose 3 (smaller), and 3 is worse than 5 for MAX, MAX will never choose this branch.

So we stop exploring that branch.

This saves computation.

4. Alpha-Beta Algorithm Steps

1. Initialize:
 - $\alpha = -\infty$
 - $\beta = +\infty$
2. If MAX node:
 - Update α
 - Prune if $\alpha \geq \beta$
3. If MIN node:
 - Update β
 - Prune if $\alpha \geq \beta$

5. Pseudocode

FUNCTION ALPHA-BETA-DECISION(STATE) RETURNS AN ACTION

RETURN $\operatorname{argmax}_{a \in \text{ACTIONS}(\text{STATE})} \text{MIN-VALUE}(\text{RESULT}(\text{STATE}, a), \alpha = -\infty, \beta = +\infty)$

FUNCTION MAX-VALUE(STATE, α , β) RETURNS A UTILITY VALUE

IF $\text{TERMINAL-TEST}(\text{STATE})$ THEN

RETURN $\text{UTILITY}(\text{STATE})$

$v \leftarrow -\infty$

FOR EACH ACTION a IN $\text{ACTIONS}(\text{STATE})$ DO

$v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(\text{STATE}, a), \alpha, \beta))$

IF $v \geq \beta$ THEN

RETURN v // beta cutoff

$\alpha \leftarrow \text{MAX}(\alpha, v)$

RETURN v

FUNCTION MIN-VALUE(STATE, α , β) RETURNS A UTILITY VALUE

IF $\text{TERMINAL-TEST}(\text{STATE})$ THEN

RETURN $\text{UTILITY}(\text{STATE})$

$v \leftarrow +\infty$

FOR EACH ACTION a IN $\text{ACTIONS}(\text{STATE})$ DO

$v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(\text{STATE}, a), \alpha, \beta))$

```

IF  $v \leq \alpha$  THEN
    RETURN  $v$     // alpha cutoff
 $\beta \leftarrow \text{MIN}(\beta, v)$ 
RETURN  $v$ 

```

6. Time Complexity

Worst Case:

$O(b^d)$

Best Case:

$O(b^{(d/2)})$

In best case, effective search depth doubles.

Performance depends heavily on move ordering.

7. Types of Cutoffs

Alpha Cutoff:

Occurs at MIN node when $\alpha \geq \beta$.

Beta Cutoff:

Occurs at MAX node when $\alpha \geq \beta$.

8. Example Code – Alpha–Beta

Python

```

import tkinter as tk
from tkinter import messagebox
import math

board = [" " for _ in range(9)]
human = "X"
ai = "O"

def check_winner(player):
    wins = [
        [0,1,2],[3,4,5],[6,7,8],
        [0,3,6],[1,4,7],[2,5,8],
        [0,4,8],[2,4,6]
    ]
    for w in wins:
        if board[w[0]] == board[w[1]] == board[w[2]] == player:
            return True
    return False

```

```

def is_full():
    return " " not in board

def minimax(is_maximizing):
    if check_winner(ai):
        return 1
    if check_winner(human):
        return -1
    if is_full():
        return 0

    if is_maximizing:
        best = -math.inf
        for i in range(9):
            if board[i] == " ":
                board[i] = ai
                score = minimax(False)
                board[i] = " "
                best = max(best, score)
        return best
    else:
        best = math.inf
        for i in range(9):
            if board[i] == " ":
                board[i] = human
                score = minimax(True)
                board[i] = " "
                best = min(best, score)
        return best

def best_move():
    best_score = -math.inf
    move = None
    for i in range(9):
        if board[i] == " ":
            board[i] = ai
            score = minimax(False)
            board[i] = " "
            if score > best_score:
                best_score = score
                move = i
    return move

def click(i):
    if board[i] == " ":
        board[i] = human
        buttons[i]["text"] = human

```

```

    if check_winner(human):
        messagebox.showinfo("Game Over", "You Win!")
        root.destroy()
        return

    if is_full():
        messagebox.showinfo("Game Over", "Draw!")
        root.destroy()
        return

    ai_move = best_move()
    if ai_move is not None:
        board[ai_move] = ai
        buttons[ai_move]["text"] = ai

        if check_winner(ai):
            messagebox.showinfo("Game Over", "AI Wins!")
            root.destroy()
            return

        if is_full():
            messagebox.showinfo("Game Over", "Draw!")
            root.destroy()

root = tk.Tk()
root.title("Tic Tac Toe - Minimax")

buttons = []
for i in range(9):
    btn = tk.Button(root, text=" ", font=("Arial", 30),
                    width=5, height=2,
                    command=lambda i=i: click(i))
    btn.grid(row=i//3, column=i%3)
    buttons.append(btn)

root.mainloop()

```

PART 3: COMPARISON

Feature	Minimax Alpha–Beta	
Explores full tree	Yes	No
Pruning	No	Yes
Speed	Slow	Faster
Optimal result	Yes	Yes
Complexity	$O(b^d)$	$O(b^{(d/2)})$ best case

PART 4: Key Theoretical Points

Minimax is:

- A depth-first search algorithm.
- A complete algorithm.
- An optimal algorithm (if full search).
- Deterministic.
- Used in adversarial search.

Alpha–Beta is:

- An optimization of Minimax.
- A pruning technique.
- Does not change correctness.
- Improves efficiency.
- Depends on move ordering.